

Universitatea "POLITEHNICA" Timișoara

Departamentul de ID/IFR și educație digitală (DeL)

Informatică

PROIECT SINCRETIC

Urmărirea inteligentă a cărților în bibliotecă

Student: Secetă Alexandru-Adrian

CUPRINS

1. Introducere
2. Descrierea proiectului
3. Analiza SWOT
4. Arhitectura sistemului (MQTT)
 - 4.1 Client embedded
 - 4.2 Broker MQTT
 - 4.3 Clienți receptori
 - 4.4 Baza de date
5. Funcții majore pe componente
6. Baza de date – tabele și relații
7. Concluzii
8. Bibliografie
9. Link GitHub (cod sursă)

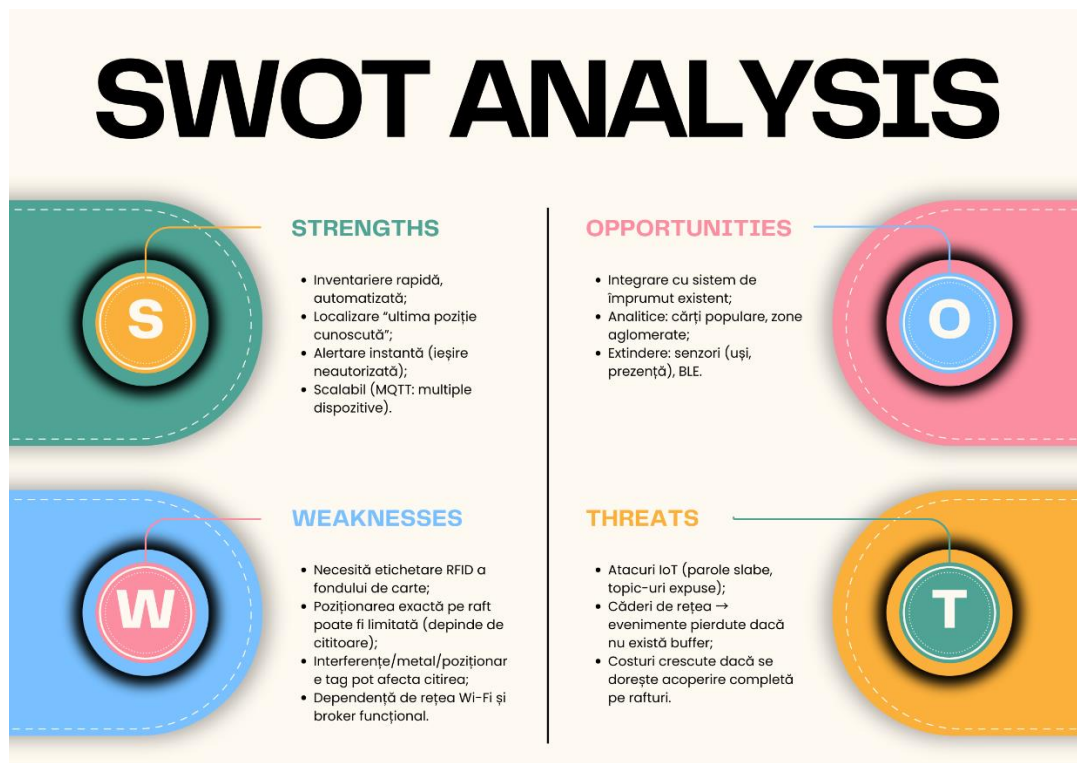
1. Introducere

Proiectul realizează un sistem IoT pentru **urmărirea inteligentă a cărților într-o bibliotecă**, astfel încât biblioteca să știe în timp real: unde se află o carte (raft / zonă / punct de împrumut), când a fost mișcată, dacă o carte a fost scoasă neautorizat dintr-o zonă, ce cărți lipsesc la inventar și unde au fost văzute ultima dată.

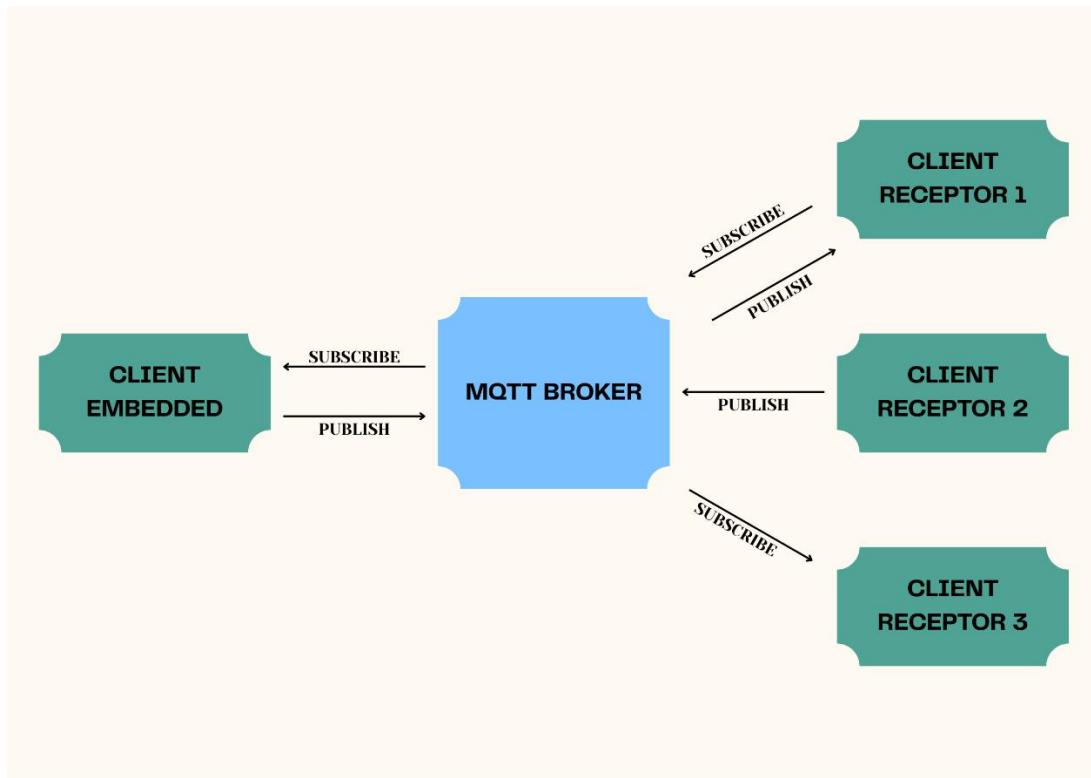
2. Descrierea proiectului

- Fiecare carte are un **tag RFID/NFC** (UID unic).
- În bibliotecă există **puncte de scanare** (dispozitive embedded) la:
 - intrare/ieșire (anti-pierdere / securitate),
 - biroul de împrumut,
 - zone/rafturi (inventariere rapidă).
- Dispozitivele embedded trimit evenimente prin **MQTT** către un broker.
- Un serviciu backend (client receptor) salvează evenimentele într-o **bază de date**.
- O aplicație web / dashboard (client receptor) afișează starea cărților și rapoarte.

3. Analiza SWOT



4. Arhitectura sistemului



4.1 Clientul embedded (dispozitivul IoT din bibliotecă)

Rol: citește tag-urile cărților și publică evenimente.

Hardware recomandat (exemplu realist de laborator):

- **ESP32** (Wi-Fi integrat)
- Modul RFID: **MFRC522** (13.56 MHz) sau alt modul compatibil
- LED + buzzer (feedback local)
- (opțional) buton “scan mode” / “inventar”

Date colectate (payload):

- tag_uid (UID RFID carte)
- device_id (ID dispozitiv)
- zone_id / shelf_id (zona în care e montat)
- event_type (SCAN, CHECKOUT, CHECKIN, EXIT_GATE)
- timestamp
- (opțional) intensitate semnal / calitate citire

Publicare MQTT (exemplu topic-uri):

- library/events/scan
- library/events/checkout
- library/events/exit_gate
- library/devices/<device_id>/status

Abonare MQTT (comenzi):

- library/devices/<device_id>/cmd (ex: “beep”, “restart”, “setZone”)

4.2 Brokerul MQTT

Rol: intermediarul care primește mesajele și le distribuie clienților abonați.

Implementare posibilă:

- Eclipse **Mosquitto** pe Raspberry Pi / PC / VM / cloud.

Funcții cheie:

- autentificare user/parolă
- ACL pe topic-uri (ex: device-urile nu pot scrie în topic-uri de admin)
- QoS (0/1) pentru fiabilitate
- retained messages pentru “last known status”
- Last Will (LWT) pentru detectarea dispozitiv offline

4.3 Clientul / Clienții receptori

Rol: prelucrează datele, declanșează alerte, afișează informații.

Exemple de clienți receptori:

- a) **Serviciu backend “Ingestor”** (Python/Node.js)
 - a. abonat la library/events/#
 - b. validează payload, atașează metadata și scrie în DB
- b) **Dashboard web** (ex: Node-RED Dashboard / aplicație web)
 - a. afișează harta zonelor și ultima locație a cărții
 - b. rapoarte: lipsuri, scanări pe zi, împrumuturi
- c) **Aplicație mobilă pentru bibliotecar**
 - a. caută carte: “unde a fost văzută ultima dată?”
 - b. primește notificări (alertă ieșire neautorizată)

4.4 Baza de date

Rol: stochează persistent evenimentele și starea curentă a cărților.

Tehnologie recomandată: PostgreSQL / MySQL (relațional, ușor de modelat entități: cărți, locații, împrumuturi).

5. Funcții majore pe componente

Funcții client embedded

- Citire tag RFID și filtrare (anti-duplicat la scanări repetate)
- Conectare Wi-Fi + conectare MQTT
- Publicare eveniment la detectarea tag-ului
- Trimitere heartbeat / status (online, RSSI Wi-Fi, uptime)
- Recepție comenzi (beep/LED/restart/setare zonă)
- Buffer local (opțional): dacă pică Wi-Fi, reține N evenimente și le trimite când revine conexiunea

Funcții broker MQTT

- Rutare mesaje publish/subscribe pe topic-uri
- Control acces (ACL)
- QoS și sesiuni persistente (după caz)
- LWT pentru detectare offline

Funcții client receptor – backend ingestor

- Subscriere library/events/#
- Validare schema JSON
- Inserare în DB: eveniment + actualizare “ultima locație”
- Detectare reguli:
 - dacă event_type=EXIT_GATE și cartea nu e “împrumutată” → alertă
- API REST (opțional) pentru dashboard (căutare carte, istoricul ei)

Funcții client receptor – dashboard

- Căutare carte după titlu/autor/cod intern
- Afișare “ultima locație cunoscută”
- Istoric mișcări (timeline)
- Lista alertelor (neautorizat / lipsă la inventar)
- Rapoarte (scanări, împrumuturi, top cărți)

6. Baza de date – tabele și relații

Tabele:

1) *books*

- *book_id* (PK)
- *title*
- *author*
- *isbn* (nullable)
- *inventory_code* (cod intern bibliotecă)
- *category*
- *status* (AVAILABLE / LOANED / LOST / MAINTENANCE)

2) *rfid_tags*

- *tag_uid* (PK) – UID RFID
- *book_id* (FK – books)
- *assigned_at*
- *active* (bool)

3) *locations* (zone/rafturi)

- *location_id* (PK)
- *name* (ex: “Raft A3”, “Birou împrumut”, “Poartă ieșire”)
- *type* (SHELF / DESK / GATE / STORAGE)
- *parent_id* (FK – locations, pentru ierarhie: sală – raft – poliță)

4) *devices*

- *device_id* (PK)
- *device_type* (SHELF_SCANNER / GATE_SCANNER / DESK_SCANNER)
- *location_id* (FK – locations)

- *last_seen*
- *firmware_version*

5) **events** (toate scanările)

- *event_id* (PK)
- *timestamp*
- *tag_uid* (FK – rfid_tags)
- *device_id* (FK – devices)
- *location_id* (FK – locations)
- *event_type* (SCAN / CHECKOUT / CHECKIN / EXIT_GATE)
- *payload_json* (text/jsonb – păstrează date extra)

6) **users** (bibliotecari / conturi)

- *user_id* (PK)
- *name*
- *role* (ADMIN / LIBRARIAN)
- *email*
- *password_hash*

7) **loans** (împrumuturi)

- *loan_id* (PK)
- *book_id* (FK – books)
- *borrower_name* (sau *borrower_id* dacă există sistem separat)
- *start_date*
- *due_date*
- *return_date* (nullable)
- *status* (OPEN / CLOSED / LATE)

8) **alerts**

- *alert_id* (PK)
- *timestamp*
- *severity* (INFO/WARN/CRITICAL)
- *type* (UNAUTHORIZED_EXIT / OVERDUE / MISSING)
- *book_id* (FK – books, nullable)
- *event_id* (FK – events, nullable)

- *resolved* (bool)
- *resolved_by* (FK – users, nullable)

Relații:

- *books 1-1..n rfid_tags* (o carte poate avea un tag activ; istoric posibil)
- *devices n-1 locations*
- *events n-1 devices, events n-1 rfid_tags, events n-1 locations*
- *loans n-1 books*
- *alerts n-1 books și/sau alerts n-1 events*

7. Concluzii

Sistemul propus îmbină un flux IoT tipic industrial (dispozitive embedded + MQTT + servicii receptoare + DB) pentru un caz real: gestiunea și securizarea fondului de carte. Soluția este scalabilă: se pot adăuga rapid noi scanere în zone fără a schimba logica centrală, deoarece MQTT permite publish/subscribe pe topic-uri dedicate. În plus, baza de date păstrează istoric complet pentru audit și rapoarte.

8. Bibliografie

- **OASIS**

MQTT Version 3.1.1 – Protocol Specification, OASIS Standard, 2014.

Disponibil la: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>

(standardul oficial utilizat pentru comunicarea dintre dispozitivele IoT și broker)

- **Eclipse Foundation**

Mosquitto – MQTT Broker Documentation, Eclipse Mosquitto Project.

Disponibil la: <https://mosquitto.org/documentation/>

(brokerul MQTT folosit în arhitectura proiectului pentru transmiterea mesajelor)

- **Espressif Systems**

ESP32 Series – Datasheet and Technical Reference Manual, Espressif, 2023.

Disponibil la: <https://www.espressif.com/en/products/socs/esp32>

(microcontrollerul utilizat ca client embedded pentru citirea tag-urilor RFID și transmiterea datelor)

- **NXP Semiconductors**

MFRC522 – RFID Reader IC Datasheet, NXP, 2016.

Disponibil la: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>

(modul RFID utilizat pentru identificarea unică a cărților din bibliotecă)

- **HiveMQ GmbH**

MQTT Essentials – Understanding QoS, Topics and Retained Messages,

Disponibil la: <https://www.hivemq.com/mqtt-essentials/>

(referință practică pentru proiectarea arhitecturii MQTT folosite în sistem)

- **PostgreSQL Global Development Group**

PostgreSQL Documentation – Database Design and SQL Reference,

Disponibil la: <https://www.postgresql.org/docs/>

(utilizată pentru proiectarea bazei de date relaționale a sistemului de urmărire a cărților)

- **Node-RED Project**

Node-RED User Guide – MQTT Integration,

Disponibil la: <https://nodered.org/docs/user-guide/mqtt>

(folosit ca aplicație client receptor pentru procesarea mesajelor și afișarea informațiilor)

- **K. Ashton**

That “Internet of Things” Thing, RFID Journal, 2009.

(lucrare de referință privind utilizarea RFID și IoT în sisteme de urmărire a obiectelor)

- **Python Software Foundation**

paho-mqtt Client Library Documentation,

Disponibil la: <https://www.eclipse.org/paho/>

(bibliotecă utilizată pentru implementarea clientului receptor care salvează datele în baza de date)

9. Codul sursă prin *github*.

Codul sursă al aplicației este disponibil pe platforma GitHub la adresa:

<https://github.com/secetaalexandru-prog/smart-library-iot.git>